Cérémonie d'élicitation

> **Note terminologique :** cette page décrit la cérémonie d'élicitation et son déroulement pratique. Pour les définitions de référence des termes utilisés ici — ÉLICITER, besoin, signal, backlog, EPIC, FEATURE, STORY, BUG, RCA, partie prenante, SENSE, FORGE, CARE, DoR, DoD et rôles — consulter le Lexique / Terminologies.
>
> Les descriptions locales doivent être lues comme des règles d'usage pour cette cérémonie, sans remplacer la terminologie officielle ni la matrice RACI.

🎯 Objectif

Pourquoi cet atelier est incontournable — même pour un « petit bug »

L'atelier d'élicitation s'applique **systématiquement**, quelle que soit la taille ou l'apparente évidence du problème — y compris pour un bug mineur ou une régression « évidente ». La durée de l'atelier peut être courte, mais **il ne doit jamais être sauté**.

> Un bug évident aujourd'hui cache souvent un problème systémique qu'on n'a jamais pris le temps de comprendre.

🔍 Ce qu'on fait dans cet atelier

C'est ici qu'on :

- **Investigue** — on creuse le problème au-delà du symptôme visible.
- **Analyse** — on réalise le *Root Cause Analysis* (RCA) pour identifier la cause racine.
- **Collecte les signaux** — métriques, données de qualimétrie, historique de tickets, fréquence de récurrence, impacts terrain : on rassemble tout ce qui est nécessaire pour prendre des décisions éclairées sur la solution à adopter.

🧭 Vision produit et remise en question

À ce stade-ci, il est aussi essentiel d'avoir une **vision produit**. L'atelier d'élicitation ne sert pas seulement à comprendre le problème technique : il sert aussi à remettre en question la valeur, la priorité et la pertinence de la demande.

On doit notamment se demander :

- Quelle est l'importance réelle de cette fonctionnalité ou de ce correctif pour les utilisateurs et pour le produit ?

- Est-ce que cette fonctionnalité doit réellement être développée, maintenue ou rendue disponible ?
- Est-ce que la solution envisagée répond à un besoin produit durable ou seulement à un symptôme ponctuel ?
- Quels sont les impacts si on choisit de ne pas livrer, de retirer, de masquer ou de repousser cette fonctionnalité ?

Cette réflexion permet d'éviter de traiter automatiquement chaque demande comme quelque chose à implémenter. Parfois, la meilleure décision produit est de **ne pas développer**, de **simplifier**, de **déprioriser**, de **désactiver** ou de **cacher** une fonctionnalité.

📌 Backlog produit : EPIC / FEATURE

C'est aussi ici qu'on alimente et structure le **backlog produit** en mode **EPIC / FEATURE**, en suivant les bonnes pratiques Jira.

Une **FEATURE** représente une livraison au niveau produit. Elle doit être ouverte systématiquement comme point d'entrée pour les souhaits ou demandes provenant des ventes, du marketing, des incidents, des bugs liés ou de toute autre source de besoin produit.

Même si le besoin semble très petit, la **FEATURE demeure un requis**. Elle permet de garder une trace claire de la décision produit, de la valeur attendue, du périmètre et du contexte de livraison.

Ensuite, la FEATURE peut être :

- rattachée à une **EPIC existante**, lorsque celle-ci correspond déjà à l'objectif d'affaires visé ;
- ou associée à une **nouvelle EPIC spécifique**, lorsqu'un nouvel objectif d'affaires ou un nouveau regroupement portefeuille est nécessaire.

Dans cette logique :

- l'**EPIC** représente le niveau portefeuille ou objectif d'affaires ;
- la **FEATURE** représente le niveau livraison produit ;
- la **STORY** ou le **BUG** représente le niveau d'exécution ou de correction.

Pour faciliter la planification et la traçabilité des livraisons, on devrait aussi attacher :

- une **version majeure** à l'EPIC ;
- une **version mineure** à la FEATURE ;
- et, au besoin, une **version patch** au BUG ou à la STORY — cette dernière étant facultative selon le contexte.

⚠️

> Cette procédure peut sembler lourde ou peu importante pour un petit besoin, mais elle fait une vraie différence. Elle permet d'éviter le travail improvisé, les décisions non documentées, les surprises non souhaitées et la multiplication de demandes difficiles à relier à une intention produit claire.

Il faut aussi éviter de maintenir un **backlog produit séparé** en dehors de Jira. Le backlog produit doit rester centralisé, traçable et aligné avec les EPIC, FEATURE, STORY et BUG utilisés pour piloter la livraison.

🗂️ Fermer plutôt que supprimer : préserver la trace des décisions

Lors de l'élicitation, il est normal — et même souhaitable — de fermer certaines **EPIC** ou **FEATURE**. En avançant dans l'analyse, on peut se rendre compte qu'un besoin n'est finalement pas nécessaire, qu'il n'apporte pas assez de valeur, qu'il est couvert autrement, qu'il n'est plus aligné avec la vision produit ou qu'il comporte trop de risques par rapport aux bénéfices attendus.

L'important est de **ne pas supprimer** ces éléments. Les fermer permet de conserver la trace du contexte, du raisonnement et du pourquoi derrière la décision.

Cette traçabilité est essentielle : si le sujet revient plus tard, l'équipe pourra comprendre ce qui avait été décidé, réévaluer la situation avec de nouvelles informations, prendre une autre décision si le contexte a changé, ou confirmer et préserver la décision initiale.

> **Bonne pratique :** lorsqu'une EPIC ou une FEATURE est fermée, documenter clairement la raison de fermeture, les hypothèses considérées, les signaux utilisés et les impacts connus. Une décision fermée mais documentée vaut beaucoup mieux qu'un élément supprimé sans historique.

👥 Participants recommandés

L'atelier d'élicitation ne devrait pas être porté par un seul rôle. Pour éviter les angles morts, on implique les bons représentants selon le contexte du besoin.

- **Product Owner / Product Manager** — porte la vision produit, la valeur et la priorisation.
- **Développement / architecture** — aide à comprendre les contraintes techniques, les impacts et les options possibles, sans transformer trop tôt l'atelier en conception détaillée.
- **QA / qualité** — apporte les risques, les spécifications de validation IF... THEN..., les régressions possibles et les signaux de qualité.
- **Support / CARE** — apporte le contexte terrain, les incidents, les irritants récurrents et l'impact client.

- **Représentants SENSE, FORGE et CARE** — à impliquer lorsque le sujet touche plusieurs phases du SDLC ou nécessite une vision complète du cycle de vie.

> **Principe :** plus le besoin est ambigu, risqué ou transversal, plus il faut élargir la participation. Pour un petit bug clair, l'atelier peut être très court et impliquer moins de personnes, mais il doit quand même exister.

📥 Inputs / Prérequis

Avant de commencer l'atelier, on devrait rassembler les informations minimales permettant de comprendre le contexte et d'éviter une discussion basée uniquement sur des impressions.

- La demande, le ticket, le bug, l'incident ou le souhait initial.
- La source du besoin : client, support, ventes, marketing, métriques, rétrospective, incident, dette technique, etc.
- Les signaux disponibles : fréquence, impact, criticité, tendances, métriques de qualité, historique des tickets, versions touchées.
- Le contexte produit : utilisateurs concernés, valeur attendue, alignement avec la roadmap, alternatives existantes.
- Les décisions passées liées au même sujet, incluant les EPIC ou FEATURE fermées.

📤 Outputs / Livrables attendus

À la fin de l'atelier, on doit pouvoir comprendre ce qui a été appris, ce qui a été décidé et quelle est la prochaine étape.

- Un besoin brut compris, reformulé et documenté.
- La source du besoin et les signaux utilisés pour appuyer l'analyse.
- Une décision produit : poursuivre, fermer, repousser, simplifier, désactiver ou investiguer davantage.
- Une FEATURE créée ou mise à jour dans Jira, puis rattachée à une EPIC existante ou à une nouvelle EPIC lorsque nécessaire.
- Un RCA ou une hypothèse de cause racine lorsque le sujet est lié à un bug, un incident ou une régression.
- Les questions ouvertes, risques, dépendances et prochaines actions.

⏱️ Durée, format et sous-phases recommandées

La durée de l'atelier doit être proportionnelle à l'ambiguïté, au risque et à l'impact du sujet. L'objectif n'est pas de créer une cérémonie lourde, mais de s'assurer que l'élicitation a réellement eu lieu.

L'atelier peut aussi être réalisé en **deux sous-phases** afin d'éviter de rassembler trop de personnes trop tôt, surtout lorsque seulement une ou deux personnes ont réellement de la matière à travailler au début.

1. **Raffinement produit initial** — une première sous-phase portée principalement par les responsables produit. On clarifie le besoin brut, la valeur attendue, l'alignement avec la vision produit, les signaux disponibles et la pertinence de poursuivre ou non.
2. **Élicitation avec les "amigos" pertinents** — une seconde sous-phase avec les bons participants selon le cas : développement, QA, architecture, support, CARE, SENSE, FORGE ou autres experts nécessaires. L'objectif est d'approfondir seulement avec les personnes utiles au sujet.

Cette approche rend l'atelier plus efficace : elle évite de mobiliser tout le monde alors que le besoin n'est pas encore assez préparé, et elle réduit les rencontres où une ou deux personnes travaillent pendant que le reste du groupe attend.

- **Bug simple ou régression évidente :** 15 à 30 minutes peuvent suffire.
- **FEATURE simple :** 30 à 60 minutes pour clarifier le besoin, la valeur et les impacts.
- **FEATURE complexe, incident majeur ou sujet transversal :** un atelier plus long ou plusieurs séances peuvent être nécessaires.

Le format peut être synchrone ou asynchrone, mais la décision finale doit être documentée dans Jira et rester traçable.

> **Une FEATURE peut rester longtemps en ÉLICITÉ.** C'est normal, surtout lorsqu'on doit expérimenter, explorer différentes options, réaliser une preuve de concept, valider des hypothèses techniques ou produit, ou aller chercher davantage de signaux avant de décider.
>
> Dans ce cas, l'objectif n'est pas de forcer rapidement la FEATURE vers des STORY prêtes à implémenter. L'objectif est plutôt d'obtenir le plus d'information possible afin de prendre une décision éclairée : poursuivre, ajuster, découper, repousser, fermer ou passer à DÉFINIR.

Le travail réalisé pendant cette phase doit être reconnu comme du **vrai travail d'équipe**. Même s'il ne s'agit pas encore de développement directement implémentable, l'investigation, les expérimentations, les preuves de concept, l'analyse technique, la collecte de métriques et la clarification produit consomment du temps et de la capacité.

Pour les équipes qui fonctionnent en sprint, ce travail devrait donc apparaître dans le sprint et être comptabilisé dans la vélocité ou la capacité des développeurs lorsqu'ils y contribuent.

À retenir : même lorsqu'une EPIC ou une FEATURE n'est pas encore prête à être découpée en STORY, il y a déjà beaucoup de travail utile dès le jour 1. Ce travail ne manque pas : il faut simplement le rendre visible et le planifier correctement.

Reconnaître ce travail évite de mettre une pression artificielle sur l'équipe pour "tourner les coins ronds" simplement afin de nourrir les développeurs avec des tâches d'implémentation. Mieux vaut investir le bon effort en ÉLICITÉ que créer trop tôt des STORY mal comprises, mal cadrées ou prématurées.

🔄 Déroulement suggéré

1. **Présenter le contexte** — rappeler la demande, la source, les utilisateurs touchés et les impacts connus.
2. **Consulter les signaux** — regarder les métriques, l'historique, la fréquence, la criticité et les données disponibles.
3. **Investiguer et analyser** — creuser le problème, réaliser le RCA au besoin et distinguer le symptôme de la cause racine.
4. **Remettre en question la valeur produit** — valider si le besoin mérite d'être livré, maintenu, repoussé, simplifié ou fermé.
5. **Structurer dans Jira** — créer ou mettre à jour la FEATURE, la rattacher à une EPIC et documenter les versions pertinentes.
6. **Décider de la suite** — poursuivre vers DÉFINIR, fermer, repousser, investiguer davantage ou déclencher un plan de réaction.
7. **Documenter** — consigner le pourquoi, les signaux, les hypothèses, les risques, les questions ouvertes et les prochaines actions.

➡️ Quand passer à DÉFINIR ?

On peut passer à la phase **DÉFINIR** lorsque le besoin est suffisamment compris pour être formalisé, sans être encore transformé prématurément en solution détaillée.

- Le besoin est reformulé et validé avec la source, l'utilisateur ou le spécialiste concerné.
- La valeur produit, l'impact et la priorité sont suffisamment clairs.
- Les signaux importants ont été consultés et documentés.
- La décision de poursuivre est explicite et traçable.
- La FEATURE existe dans Jira et est rattachée correctement au backlog produit.
- Les questions ouvertes restantes sont connues et ne bloquent pas l'entrée en DÉFINIR.

✅

Critère simple : on peut passer à DÉFINIR quand l'équipe comprend assez bien le problème, la valeur et le contexte pour commencer à formaliser le besoin sans deviner.

📊 Exemple concret : la régression récurrente

Par exemple : *cette régression s'est produite 138 fois cette année*. Un tel constat change complètement la nature de la réponse. Même si on doit systématiquement résoudre chaque problème, l'élicitation permet de déterminer le niveau d'investigation requis et la meilleure stratégie à adopter.

Pour chaque résolution, on devrait viser trois niveaux de réponse :

1. **Tactique de prévention** — une mesure concrète à implémenter pour éviter que la régression ne se reproduise, par exemple un test automatisé, un garde-fou ou une validation supplémentaire.
2. **Alerte ou détection** — un mécanisme pour détecter rapidement si la régression réapparaît malgré la prévention, par exemple une alerte, un indicateur ou un test de non-régression ciblé.
3. **Plan de réaction** — une procédure documentée au cas où la régression surviendrait à nouveau, par exemple un rollback, un correctif rapide ou une communication client.

⚠️ Le piège principal : sauter les phases

Le piège le plus fréquent est de **sauter ÉLICITER, DÉFINIR et PLANIFIER** pour aller directement à l'implémentation.

> « Ça va plus vite si on code tout de suite… »

En réalité, c'est souvent l'inverse. Sauter ces phases génère des corrections incomplètes ou mal ciblées, crée de nouvelles régressions ou de la dette technique, et oblige à revenir en arrière. On finit alors par **ralentir** au lieu d'être plus rapide.

Règle d'or : même si la phase est courte, elle doit exister. Un atelier d'élicitation de 15 minutes pour un bug simple vaut toujours mieux que zéro élicitation.

📝 Historique et traces des sessions

Les ateliers d'élicitation réels peuvent être documentés comme pages enfants sous cette cérémonie afin de conserver une trace des discussions, décisions, signaux analysés et retours vers les phases appropriées.

Chaque session devrait idéalement utiliser un gabarit commun pour faciliter la relecture, le suivi et l'amélioration continue.

- **Date et contexte :** besoin, bug, incident, souhait client, demande interne, EPIC ou FEATURE concernée.
- **Participants :** rôles présents et expertises consultées.
- **Source du besoin :** client, support, ventes, marketing, métriques, qualimétrie, rétrospective, incident ou dette technique.
- **Signaux analysés :** fréquence, impact, criticité, historique, métriques, versions touchées, retours terrain.
- **Analyse / RCA :** symptômes observés, hypothèses, cause racine connue ou piste d'investigation.
- **Décision produit :** poursuivre, fermer, repousser, simplifier, désactiver, cacher ou investiguer davantage.
- **Structure Jira :** EPIC, FEATURE, STORY ou BUG créés, mis à jour, fermés ou reliés.
- **Actions et propriétaires :** prochaines étapes, responsables et échéances.
- **Retours vers les phases :** éléments à envoyer vers DÉFINIR, PLANIFIER, IMPLÉMENTER, DIFFUSER, CONFIRMER ou vers la qualimétrie.
- **Conclusion :** prêt pour DÉFINIR, à investiguer davantage, à fermer, à replanifier ou à escalader.

📚 Références

- Comment éliciter
- Cérémonie de définition — cérémonie suivante
- (SDLC) Software Development Life Cycle
- Contrat qualité Opal-RT
- RACI — Rôles et responsabilités
- Qualimétrie
- Lexique / Terminologies